

PQ-CAN: A Framework for Simulating Post-Quantum Cryptography in Embedded Systems

Mauro Conti
Department of Mathematics
University of Padova
Padova, Italy
mauro.conti@unipd.it

Francesco Marchiori
Department of Mathematics
University of Padova
Padova, Italy
francesco.marchiori@math.unipd.it

Sebastiano Matarazzo
Department of Mathematics
University of Padova
Padova, Italy
sebastiano.matarazzo@studenti.unipd.it

Marco Rubin
Department of Mathematics
University of Padova
Padova, Italy
marco.rubin.5@studenti.unipd.it

Abstract—The rapid development of quantum computers threatens traditional cryptographic schemes, prompting the need for Post-Quantum Cryptography (PQC). Although the NIST standardization process has accelerated the development of such algorithms, their application in resource-constrained environments such as embedded systems remains a challenge. Automotive systems relying on the Controller Area Network (CAN) bus for communication are particularly vulnerable due to their limited computational capabilities, high traffic, and need for real-time response. These constraints raise concerns about the feasibility of implementing PQC in automotive environments, where legacy hardware and bit rate limitations must also be considered.

In this paper, we introduce PQ-CAN, a modular framework for simulating the performance and overhead of PQC algorithms in embedded systems. We consider the automotive domain as our case study, testing a variety of PQC schemes under different scenarios. Our simulation enables the adjustment of embedded system computational capabilities and CAN bus bit rate constraints. We also provide insights into the trade-offs involved by analyzing each algorithm's security level and overhead for key encapsulation and digital signature. By evaluating the performance of these algorithms, we provide insights into their feasibility and identify the strengths and limitations of PQC in securing automotive communications in the post-quantum era.

Index Terms—Post-Quantum Cryptography, Embedded Systems, Automotive, CAN bus

I. INTRODUCTION

The rapid progress of quantum computing technologies presents a profound threat to traditional cryptographic systems at the backbone of modern communications. As quantum computers advance, they threaten the security of widely used cryptographic algorithms, particularly those based on integer factorization [1] and discrete logarithms [2], which are vulnerable to Shor's and Grover's algorithms [3], [4]. In response, Post-Quantum Cryptography (PQC) has emerged as a crucial field, offering algorithms designed to resist quantum-based attacks by relying on computationally hard mathematical problems, such as lattice-based, code-based, and multivariate polynomial challenges. The importance of PQC has been further emphasized by the ongoing National Institute

of Standards and Technology (NIST) standardization process, which aims to establish cryptographic algorithms that can withstand quantum attacks while remaining practical for real-world deployment [5].

Despite the progress in PQC standardization, integrating these algorithms into resource-constrained environments, such as embedded systems, remains a significant challenge. Indeed, embedded devices, including those used in industrial control systems, Internet of Things (IoT), and automotive applications, often have strict limitations on computational power, memory, and energy consumption [6]. Instead, PQC algorithms demand substantial computational resources, increased memory for key storage, and higher bandwidth for transmitting larger cryptographic primitives, posing integration challenges in such constrained environments. One critical domain where these constraints are particularly evident is the automotive industry, where the Controller Area Network (CAN) bus serves as the backbone of in-vehicle communication. Designed for efficiency and reliability, the CAN bus lacks built-in cryptographic protections, making it vulnerable to conventional and quantum-era attacks [7]. Implementing PQC in this environment requires careful consideration of algorithmic overhead, latency, and real-time constraints to ensure secure and efficient communication between embedded systems in automotive electronics, called Electronic Control Units (ECUs). Understanding the trade-offs between security and performance is essential for assessing the feasibility of PQC in securing automotive networks against future threats. This raises a critical question: *can PQC be implemented in embedded systems with varying constraints?* In particular, we identify the following research questions.

- RQ1** Can PQC be implemented in existing lower-end embedded devices?
- RQ2** Can PQC be implemented in time-sensitive applications?
- RQ3** What is the trade-off between computational overhead and security level with PQC in embedded systems?

Contribution. In this paper, we present **PQ-CAN**, a modular and comprehensive framework designed to evaluate the feasibility of PQC algorithms in embedded systems. Using the automotive domain as a case study, we analyze several PQC algorithms under varying computational capabilities to assess their practical application. Our findings highlight a critical trade-off between security strength and computational overhead, demonstrating the challenges of implementing specific algorithms in legacy embedded systems. Built with scalability in mind, PQ-CAN is implemented in Docker, allowing it to simulate a wide range of embedded systems not limited to the automotive domain. Our contributions can be summarized as follows.

- We introduce PQ-CAN, a novel, modular framework for evaluating PQC algorithms in embedded systems.
- We present a detailed study of PQC algorithms in the automotive domain, offering insights into their feasibility under different system constraints. In particular, we focus on schemes pertaining to Key Encapsulation Mechanisms (KEMs) and Digital Signature Algorithms (DSAs).
- We make PQ-CAN open-source, providing public access at: <https://anonymous.4open.science/r/PQ-CAN-1518>.

Organization. The remainder of this paper is organized as follows. Section II reviews background knowledge and related works on PQC and automotive systems. We report our methodology in Section III, and we show the results of our evaluation in Section IV. Finally, we provide our discussion in Section V, and Section VI concludes our work.

II. RELATED WORKS

In this section, we review related works on PQC (Section II-A) and CAN bus security (Section II-B).

A. Post-Quantum Cryptography

PQC aims to secure communications against quantum attacks, primarily using lattice-based, code-based, and multivariate polynomial cryptography. Lattice-based schemes such as CRYSTALS-KYBER [8] and CRYSTALS-Dilithium [9] are favored for their efficiency and security, while FALCON [10] and SPHINCS⁺ [11] offer alternative signature mechanisms with varying trade-offs. However, these algorithms demand significant computational resources, making them challenging for resource-constrained environments like embedded systems. To address this, research has focused on optimizing these schemes for lightweight applications [12]. Still, balancing security and computational overhead remains a challenge [13].

B. CAN Bus

The CAN bus is a multi-master, message-based communication protocol developed by Robert Bosch GmbH in the early 1980s [7]. It was designed to exchange data efficiently between ECUs, and due to its fault tolerance, high reliability, and lightweight communication overhead, CAN has become a widely adopted standard across multiple industries, including automotive. A CAN network uses a two-wire differential

signaling scheme for electromagnetic interference resistance. It operates at the OSI data-link layer, employing a frame-based structure with fields like an identifier, data payload, CRC, and acknowledgment bits, supporting data rates up to 1 Mbps. CAN enables broadcast communication between ECUs, which manage most vehicle functions, ranging from critical real-time operations to infotainment, and can vary significantly in computational capabilities based on their role. Despite its efficiency, CAN lacks fundamental security mechanisms such as authentication, encryption, and access control [14]. As a result, research has increasingly focused on implementing authentication mechanisms to enhance its security [15].

III. METHODOLOGY

In this section, we report our proposed methodology for simulating PQC algorithms in embedded systems. We overview our simulation environment in Section III-A, and we discuss the considered algorithms in Section III-B.

A. Simulation

Our simulation framework provides a flexible and scalable environment for evaluating PQC in embedded systems. It is built on a containerized architecture, representing each embedded device as an isolated Docker container. This approach allows for the simulation of different network topologies, hardware constraints, and communication protocols, making it adaptable to various embedded systems beyond the automotive sector. To accurately model embedded constraints, our framework supports:

- CPU frequency constraints to emulate different processing capabilities.
- Bit rate limitations to impose real-world communication delays.
- Traffic injection mechanisms to simulate network congestion and interference.

For our study, we apply this framework to an automotive CAN bus 2.0 environment to analyze the feasibility of PQC in vehicle networks. Each ECU container runs an isolated cryptographic process in this context, with all containers sharing the same network namespace. Among these, Alice and Bob represent the two ECU implementing the cryptographic communication. The implementations adhere closely to NIST's C language API conventions,¹ with only minor deviations. We then develop different programs for Alice and Bob implementing cryptographic schemes and linking them to a separate cryptographic library optimized with AVX-2 instructions. The communication happens on a virtual CAN bus, whose interface is offered by the `vxcan` driver, which creates two ends of a communication link (one in the host, the other in the network namespace of the containers). The host can set the bit rate for both ends, e.g., via the `tc` utility of the `iproute2` suite, and the Linux kernel will manage the traffic so as not to exceed the requested limit. Hyper-Threading and Turbo Boost are disabled, and two cores are

¹<https://csrc.nist.gov/projects/post-quantum-cryptography/pqc-archive>

reserved for Alice and Bob through the `taskset` utility from the `util-linux` package. We also fix the clock frequency of these cores via Docker's `cpus` flag, which constrains the containers to a maximum usage of the host's CPU cycles (i.e., by setting it to the fraction of the target frequency over the host frequency, as `target_freq/host_freq`).

B. Cryptographic Algorithms

This work evaluates seven post-quantum cryptographic schemes, focusing on KEM (Fig. 1a) and DSA (Fig. 1b) algorithms. For KEM, we consider CRYSTALS-KYBER, BIKE, HQC, and Classic McEliece in Section III-B1. For DSA, we consider CRYSTALS-Dilithium, SPHINCS⁺, and FALCON in Section III-B2.

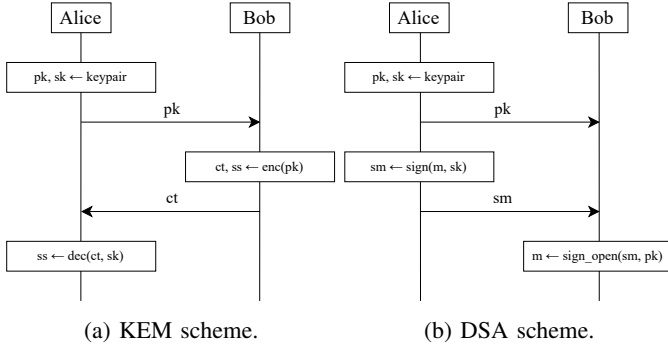


Fig. 1: Diagrams of the cryptosystems considered in this study.

1) *KEM Algorithms*: These algorithms enable secure key exchange over insecure channels by allowing two parties to establish a shared secret without directly transmitting the key. The process involves encapsulating the secret using a public key and decapsulating it with a private key, ensuring confidentiality for encrypted communication.

a) *CRYSTALS-KYBER*: A lattice-based KEM that relies on the Module Learning-With-Errors (MLWE) problem [8]. It balances security and efficiency with smaller key sizes and faster computations than other lattice-based schemes. KYBER comes in three variants: KYBER512, 768, and 1024, each providing different trade-offs between security and performance.

b) *BIKE*: A code-based KEM relying on the Quasi-Cyclic Moderate-Density Parity-Check (QC-MDPC) code problem, offering security against quantum attacks through hard-to-decode random linear codes [16]. BIKE comes in three variants: BIKE Level-1, Level-3, and Level-5, which balance security and performance, with BIKE Level-1 being the most efficient and BIKE Level-5 offering the highest security at a higher computational cost.

c) *HQC*: A code-based KEM that secures key exchange using the Syndrome Decoding Problem (SDP) in the Hamming metric [17]. It leverages Quasi-Cyclic (QC) linear codes to balance security and efficiency, though its key and ciphertext sizes are larger than lattice-based schemes. HQC offers three variants: hqc-128, hqc-192, and hqc-256—providing increasing levels of security at the cost of higher computational and storage requirements.

d) *Classic McEliece*: A code-based KEM relying on the hardness of decoding random binary Goppa codes [18]. It offers strong security, but suffers from extremely large public key sizes and challenging deployment in constrained environments. Classic McEliece provides multiple parameter sets, with higher values offering increased security at the cost of larger keys and computational overhead. “f” variants exist for faster key generation without altering security parameters.

2) *DSA Algorithms*: These algorithms ensure message authenticity and integrity through verifiable signatures, allowing a sender to sign messages with a private key and a receiver to verify them using the corresponding public key.

a) *CRYSTALS-Dilithium*: A lattice-based DSA securing signatures through the MLWE and Module Short Integer Solution (MSIS) problems [9]. It uses rejection sampling to prevent side-channel leaks and operates with integer-based arithmetic for efficiency. Dilithium offers three variants: Dilithium Level 2, Level 3, and Level 5—balancing security and performance, with higher levels providing stronger protection at the cost of larger keys and signatures.

b) *SPHINCS⁺*: A hash-based DSA offering post-quantum security without algebraic assumptions, relying solely on cryptographic hash functions [11]. It uses Merkle trees, one-time signatures, and few-time signatures, ensuring robustness but with larger signatures and higher computational costs. SPHINCS⁺ supports SHA-256, SHAKE256, and Haraka, with security levels of 128, 192, and 256 bits. It offers “fast” and “small” optimizations, as well as “robust” and “simple” variants, balancing security and efficiency.

c) *FALCON*: A lattice-based DSA based on the Short Integer Solution (SIS) problem over N^{th} degree Truncated polynomial Ring Units (NTRU) lattices, offering compact signatures with strong security [10]. It uses Gaussian sampling via FFT for efficient signing but requires careful implementation due to floating-point dependencies. FALCON-512 (NIST level 1) provides small signatures and fast verification, while FALCON-1024 (NIST level 5) enhances security with larger keys and signatures, balancing efficiency and robustness.

IV. EVALUATION

We now report the results of our experiments. We first overview our experimental setup in Section IV-A, and we disclose our simulation results for KEMs and DSAs in Section IV-B and Section IV-C, respectively.

A. Experimental Setup

The experimental setup employed in this study consists of a simulated CAN bus 2.0 environment running on Arch Linux 6.12.10, powered by an Intel Core i7-1065G7, with 16 GB of RAM. To represent a range of different automotive systems, we simulated three ECU configurations: a low-end, a mid-range, and a high-end one, selecting their clock frequencies based on an analysis of the hardware available on the market [19], [20]. The configurations operate with ECUs running at 120 MHz (“low”), 200 MHz (“mid”), and 300 MHz (“high”), each connected to a CAN bus supporting a maximum bit

rate of 125 Kbps, 500 Kbps, and 1 Mbps, respectively. Since Alice and Bob are not synchronized, message loss is possible, reflecting real-world CAN bus scenarios where ECUs operate asynchronously. This allows us to assess the success rate of a plain implementation of the schemes in Fig. 1. Therefore, a 2-second timeout was set on the receiver side, mimicking practical timeout mechanisms used in automotive networks to prevent indefinite waiting. To simulate background traffic, we implement another container, Charlie, loading the CAN bus to 88% capacity with the `canbusgen` utility of the `can-utils` suite. The sender has no timeout, replicating cases where ECUs experience transmission delays under heavy traffic. For each algorithm and configuration, we perform 100 iterations, measuring execution times, success rates, and overhead. Outliers due to processor frequency instability were omitted. Under heavy traffic, the long receiver timeout sometimes led to delayed packets, increasing success rates by extending the reception window. Overhead includes cryptographic operations and memory management. For KEMs, it covers key generation to decapsulation, while for DSAs, we isolated cryptographic overhead by subtracting nominal message transmission time from total execution time.

B. KEM Results

Our analysis, represented in Table I, shows that communication overhead dominates total execution time, with cryptographic operations accounting for only about 13% of it on average. Among the tested algorithms, KYBER512 consistently exhibits the lowest overhead and fastest cryptographic operations across all configurations. As security levels increase, the other KYBER variants maintain a strong balance between performance and security, outperforming alternatives with lower security levels. Among level 1 security algorithms, BIKE Level-1 is significantly slower than even the most secure KYBER variant, KYBER1024, while hqc-128 already lags behind other contenders. Higher security levels exacerbate these delays: BIKE Level-3, BIKE Level-5, hqc-192, and hqc-256 exhibit substantial execution times, with the latter reaching up to 5 seconds in the “low” performance configuration. Interestingly, security level alone does not dictate performance; public key and ciphertext sizes significantly impact communication times. More insight on this is shown in our repository. However, within each algorithm family, higher security variants generally follow expected trends in execution time. Success rates tend to drop in lower-performance configurations. Still, algorithms with longer communication times, such as hqc-192 and hqc-256, show higher success rates due to the extended reception window, as explained earlier. In contrast, faster algorithms complete transmission too quickly to benefit from this effect. We excluded Classic McEliece from our comparison due to its excessive latency, which ranged from 8 to 49 seconds even in its “f” variants under the high configuration. The variants of KYBER remain the best choice for optimal balance between speed and security, delivering execution times at least 300 times shorter than hqc-192 and hqc-256 while maintaining an acceptable success rate.

C. DSA Results

In our analysis, Alice generated a random 32-byte message using NIST’s `randombytes` function at each run, with its execution time included in the overhead. In Table II, only the NIST-standardized SPHINCS+ variants are considered, omitting the “robust” and “Haraka” versions. The “s” variants and those offering security level 5 were also excluded due to poor performance: the former had near-zero success rates, as Alice’s signing time exceeded Bob’s 2-second timeout. At the same time, the latter achieved at most a 2% success rate. The analysis shows that in FALCON-512 and FALCON-1024, where public key and signature sizes are relatively small, key generation dominates the overhead. Communication time becomes the primary contributor as key and signature sizes grow, particularly in SPHINCS+, where long signatures significantly increase signing times. Despite their higher processing times, Dilithium and SPHINCS+ suffer from lower success rates than FALCON due to synchronization issues. Indeed, Bob often fails to resume listening in time, missing Alice’s signed message and triggering the timeout. Among the schemes, FALCON-1024 has slightly higher overhead than Dilithium Level 5 in “high” and “mid” configurations but outperforms all Dilithium variants in “low”. It also achieves the highest success rates in “mid” and “low,” though in “high”, FALCON-512 performs marginally better. The SPHINCS+ variants exhibit the worst success rates and longest overheads, with some reaching nearly 10 seconds in the “low” configuration. FALCON-512 offers the best balance of low overhead and high success rate, though it remains slower than the nominal baseline. Dilithium’s lower success rates make it less ideal if stronger security is needed, while FALCON-1024 provides both high security and success rates at the cost of more significant overhead.

V. DISCUSSION

Based on the results presented in Section IV, we can conclude that PQC algorithms are feasible for implementation on lower-end and legacy embedded devices, including legacy ECUs. However, this comes with significant trade-offs regarding success rate, computational overhead, and security level. For KEM, KYBER512 shows the lowest overhead (≈ 1.126 ms) with a 59% success rate in low-end settings, while higher-security alternatives like BIKE Level-5 and hqc-256 introduce prohibitive overheads (≈ 1961 – 4921 ms). Similarly, for DSA, FALCON-512 achieves moderate overhead (≈ 167.53 ms) with a 75% success rate, whereas Dilithium Level 2 demonstrates lower overhead but limited success. Thus, lightweight PQC schemes may be viable with optimizations, but high-security implementations remain challenging.

RQ1 Takeaway – Lower-end embedded devices can support PQC, but only with low-overhead algorithms, requiring trade-offs in security or reliability and limiting their suitability for critical tasks.

PQC can also be implemented in time-sensitive applications, but their feasibility depends on the specific algorithm and

TABLE I: Comparison of the KEM schemes sorted by the lowest overhead.

Algorithm	Set	Timings [ms]				Success Rate	Security Level
		Key Generation	Encapsulation	Decapsulation	Overhead ↓		
KYBER512	high	0.045 ± 0.019	0.063 ± 0.024	0.030 ± 0.018	1.189 ± 0.583	0.93	1
	mid	0.048 ± 0.014	0.064 ± 0.011	0.032 ± 0.009	1.080 ± 0.193	0.82	
	low	0.042 ± 0.019	0.055 ± 0.023	0.028 ± 0.013	1.126 ± 0.628	0.59	
KYBER768	high	0.049 ± 0.018	0.070 ± 0.029	0.038 ± 0.020	1.459 ± 0.593	0.94	3
	mid	0.059 ± 0.018	0.074 ± 0.015	0.044 ± 0.013	1.457 ± 0.305	0.77	
	low	0.052 ± 0.021	0.068 ± 0.028	0.039 ± 0.016	1.356 ± 0.343	0.64	
KYBER1024	high	0.073 ± 0.065	0.086 ± 0.058	0.061 ± 0.037	1.998 ± 0.749	0.90	5
	mid	0.065 ± 0.021	0.091 ± 0.017	0.055 ± 0.015	1.825 ± 0.377	0.80	
	low	0.060 ± 0.029	0.078 ± 0.029	0.050 ± 0.025	1.682 ± 0.450	0.67	
BIKE Level-1	high	0.363 ± 0.188	0.141 ± 0.054	1.291 ± 0.934	3.542 ± 1.466	0.91	1
	mid	0.455 ± 0.129	0.157 ± 0.032	1.333 ± 0.329	3.583 ± 0.624	0.89	
	low	0.345 ± 0.116	0.126 ± 0.049	1.237 ± 0.706	3.242 ± 1.113	0.59	
hqc-128	high	0.073 ± 0.038	0.201 ± 0.045	0.354 ± 0.084	137.374 ± 4.645	0.96	1
	mid	0.097 ± 0.027	0.211 ± 0.010	0.316 ± 0.024	272.631 ± 9.025	0.74	
	low	0.071 ± 0.033	0.177 ± 0.060	0.291 ± 0.112	1099.459 ± 40.068	0.72	
BIKE Level-3	high	0.975 ± 0.480	0.309 ± 0.075	4.523 ± 1.187	140.559 ± 5.243	0.92	3
	mid	1.223 ± 0.363	0.322 ± 0.011	4.516 ± 0.651	274.841 ± 9.259	0.74	
	low	0.867 ± 0.403	0.260 ± 0.085	4.420 ± 1.618	1080.292 ± 39.313	0.65	
BIKE Level-5	high	2.291 ± 1.113	0.605 ± 0.141	9.702 ± 3.206	258.926 ± 11.190	0.96	5
	mid	2.977 ± 0.883	0.608 ± 0.012	11.408 ± 1.508	501.347 ± 9.840	0.88	
	low	2.184 ± 0.975	0.500 ± 0.171	10.393 ± 9.710	1961.925 ± 38.717	0.58	
hqc-192	high	0.159 ± 0.095	0.413 ± 0.113	0.699 ± 0.274	358.374 ± 14.621	0.97	3
	mid	0.175 ± 0.059	0.422 ± 0.015	0.626 ± 0.036	717.210 ± 15.302	0.80	
	low	0.140 ± 0.061	0.341 ± 0.129	0.551 ± 0.287	2728.391 ± 37.556	0.74	
hqc-256	high	0.291 ± 0.142	0.700 ± 0.231	1.312 ± 0.456	598.037 ± 8.012	0.99	5
	mid	0.352 ± 0.078	0.785 ± 0.026	1.192 ± 0.031	1234.307 ± 14.672	0.90	
	low	0.248 ± 0.109	0.598 ± 0.225	1.045 ± 0.435	4921.836 ± 32.078	0.72	

security level. For KEM, KYBER512 achieves a low overhead (≈ 1.1 ms) with a high success rate (0.93), making it suitable for real-time systems when higher-end embedded devices are available. In contrast, schemes like hqc-256 and BIKE Level-5 exhibit significantly higher overhead (<250 ms and >1000 ms, respectively), which may be impractical for strict timing constraints. For DSA, FALCON-512 maintains a low verification time (≈ 0.1 ms) but suffers from high key generation latency, while Dilithium Level 2 offers a more balanced trade-off.

RQ2 Takeaway – Low-latency PQC algorithms like KYBER512 and Dilithium Level 2 can be deployed in time-sensitive applications when higher-end ECUs are available. Still, high-security schemes with excessive overhead may pose challenges.

PQC algorithms in embedded systems present a trade-off between computational overhead and security level. Higher security levels, such as those in KYBER1024, BIKE Level-5, and hqc-256, require significantly more processing time for key generation, encapsulation, and decapsulation, leading to increased overhead. Meanwhile, lower security levels, such as KYBER512 and FALCON-512, offer reduced computational costs but a lower success rate and resilience against quantum attacks. Indeed, BIKE Level-5 exhibits high overhead

(≈ 1961 ms) but ensures high security, while KYBER512 maintains minimal overhead (≈ 1.1 ms) at the cost of lower security.

RQ3 Takeaway – Higher security in PQCs comes at the cost of significantly increased computational overhead in embedded devices, making it crucial to balance performance and security in embedded systems.

VI. CONCLUSIONS

This paper presented PQ-CAN a novel framework for the simulation of PQC algorithms in embedded systems and assessed their integration into automotive ECUs within simulated CAN bus environments. Results indicate that communication overhead is the primary performance bottleneck, with KYBER and FALCON-512 emerging as the most efficient key encapsulation and signature schemes, while Classic McEliece and SPHINCS⁺ proved impractical due to excessive execution times. Future work should focus on real-world testing on physical CAN networks, improving underperforming implementations, and exploring other PQC schemes. The framework could also be extended beyond automotive applications to other constrained environments, such as industrial control systems, avionics communication, or IoT networks relying on lightweight message protocols, broadening its applicability in post-quantum security.

TABLE II: Comparison of the DSA schemes sorted by the lowest overhead.

Algorithm	Set	Timings [ms]					Success Rate	Security Level
		Nominal	Key Generation	Signing	Verification	Overhead ↓		
FALCON-512	high	0.180 ± 0.070	43.037 ± 39.540	0.492 ± 0.237	0.089 ± 0.088	46.995 ± 40.303	0.96	1
	mid	0.178 ± 0.070	76.429 ± 53.586	0.606 ± 0.222	0.112 ± 0.080	90.810 ± 52.814	0.84	
	low	0.209 ± 0.115	139.859 ± 77.103	0.528 ± 0.243	0.123 ± 0.092	167.530 ± 85.312	0.75	
Dilithium Level 2	high	0.180 ± 0.070	0.071 ± 0.032	0.144 ± 0.094	0.117 ± 0.034	61.275 ± 7.689	0.28	2
	mid	0.178 ± 0.070	0.078 ± 0.026	0.154 ± 0.085	0.111 ± 0.011	120.438 ± 9.534	0.35	
	low	0.209 ± 0.115	0.068 ± 0.025	0.133 ± 0.092	0.090 ± 0.026	470.414 ± 38.111	0.29	
Dilithium Level 3	high	0.180 ± 0.070	0.099 ± 0.045	0.338 ± 0.181	0.158 ± 0.047	101.937 ± 5.303	0.50	3
	mid	0.178 ± 0.070	0.124 ± 0.039	0.312 ± 0.150	0.154 ± 0.010	201.297 ± 9.269	0.61	
	low	0.209 ± 0.115	0.092 ± 0.042	0.241 ± 0.117	0.143 ± 0.075	821.569 ± 48.606	0.50	
Dilithium Level 5	high	0.180 ± 0.070	0.135 ± 0.066	0.311 ± 0.169	0.206 ± 0.076	142.395 ± 5.226	0.54	5
	mid	0.178 ± 0.070	0.184 ± 0.053	0.405 ± 0.129	0.219 ± 0.016	283.322 ± 10.458	0.51	
	low	0.209 ± 0.115	0.143 ± 0.066	0.315 ± 0.166	0.186 ± 0.062	1131.983 ± 38.160	0.43	
FALCON-1024	high	0.180 ± 0.070	138.089 ± 106.749	1.015 ± 0.491	0.093 ± 0.140	159.200 ± 118.779	0.93	5
	mid	0.178 ± 0.070	265.043 ± 89.260	1.445 ± 0.198	0.171 ± 0.156	293.600 ± 123.264	0.96	
	low	0.209 ± 0.115	361.655 ± 164.442	0.921 ± 0.493	0.251 ± 0.171	388.039 ± 173.958	0.92	
SPHINCS ⁺ -128f (SHA2)	high	0.180 ± 0.070	0.784 ± 0.363	85.286 ± 58.384	1.800 ± 0.506	580.442 ± 64.255	0.21	1
	mid	0.178 ± 0.070	1.018 ± 0.271	134.187 ± 66.447	1.977 ± 0.240	1119.833 ± 66.947	0.27	
	low	0.209 ± 0.115	0.898 ± 0.644	185.383 ± 81.223	1.709 ± 0.672	4078.748 ± 103.034	0.26	
SPHINCS ⁺ -128f (SHAKE)	high	0.180 ± 0.070	0.820 ± 0.378	99.156 ± 63.127	2.036 ± 0.318	591.207 ± 52.683	0.14	1
	mid	0.178 ± 0.070	1.041 ± 0.301	144.316 ± 65.958	1.803 ± 0.064	1131.293 ± 68.882	0.23	
	low	0.209 ± 0.115	0.740 ± 0.377	163.746 ± 69.818	1.549 ± 0.794	4023.570 ± 91.915	0.18	
SPHINCS ⁺ -192f (SHA2)	high	0.180 ± 0.070	1.086 ± 0.501	104.257 ± 61.055	3.149 ± 1.157	1196.703 ± 23.207	0.04	3
	mid	0.178 ± 0.070	1.412 ± 0.421	234.477 ± 72.334	2.964 ± 0.261	2432.741 ± 42.550	0.09	
	low	0.209 ± 0.115	1.060 ± 0.503	281.709 ± 114.678	2.413 ± 0.719	9066.902 ± 123.167	0.13	
SPHINCS ⁺ -192f (SHAKE)	high	0.180 ± 0.070	1.163 ± 0.517	125.205 ± 69.049	2.838 ± 0.064	1204.315 ± 2.236	0.02	3
	mid	0.178 ± 0.070	1.385 ± 0.370	235.393 ± 67.389	2.551 ± 0.055	2328.240 ± 45.730	0.06	
	low	0.209 ± 0.115	1.049 ± 0.566	230.640 ± 71.643	2.125 ± 0.852	9243.958 ± 133.199	0.04	

REFERENCES

- [1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [2] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.
- [3] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
- [4] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, 1996, pp. 212–219.
- [5] National Institute of Standards and Technology, "Announcing approval of three federal information processing standards (FIPS) for post-quantum cryptography," Aug. 2024, Accessed: 2025-02-05. [Online]. Available: <https://csrc.nist.gov/news/2024/postquantum-cryptography-fips-approved>
- [6] T. M. Fernández-Caramés, "From pre-quantum to post-quantum IoT security: a survey on quantum-resistant cryptosystems for the Internet of Things," *IEEE Internet of Things Journal*, vol. 7, no. 7, pp. 6457–6480, 2019.
- [7] "ISO 11898:2024: Road vehicles — Controller area network (CAN)," International Organization for Standardization, Geneva, CH, Standard, May 2024.
- [8] R. Avanzi *et al.*, "Kyber," Accessed: Jan. 13, 2025. [Online]. Available: <https://pq-crystals.org/kyber/index.shtml>
- [9] L. Ducas *et al.*, "Dilithium," Accessed: Jan. 13, 2025. [Online]. Available: <https://pq-crystals.org/dilithium/index.shtml>
- [10] P.-A. Fouque *et al.*, "Falcon," Accessed: Jan. 13, 2025. [Online]. Available: <https://falcon-sign.info/>
- [11] J.-P. Aumasson *et al.*, "SPHINCS+," Accessed: Jan. 13, 2025. [Online]. Available: <https://sphincs.org/>
- [12] T. Liu, G. Ramachandran, and R. Jurdak, "Post-quantum cryptography for Internet of Things: a survey on performance and optimization," 2024. [Online]. Available: <https://arxiv.org/abs/2401.17538>
- [13] O. Kuznetsov, S. Kandi, E. Frontoni, and O. Smirnov, "Trade-offs in post-quantum cryptography: A comparative assessment of BIKE, HQC, and Classic McEliece," in *CQPC*, 2023, pp. 1–11.
- [14] O. Avatefipour and H. Malik, "State-of-the-art survey on in-vehicle network communication (CAN-bus) security and vulnerabilities," 2018. [Online]. Available: <https://arxiv.org/abs/1802.01725>
- [15] A. Lotto, F. Marchiori, A. Brighente, and M. Conti, "A survey and comparative analysis of security properties of CAN authentication protocols," *IEEE Communications Surveys & Tutorials*, pp. 1–1, 2024.
- [16] N. Aragon *et al.*, "BIKE - Bit Flipping Key Encapsulation," Accessed: Jan. 13, 2025. [Online]. Available: <https://bikesuite.org/>
- [17] C. Aguilar Melchor *et al.*, "HQC," Accessed: Jan. 13, 2025. [Online]. Available: <https://pqc-hqc.org/>
- [18] D. J. Bernstein *et al.*, "Classic McEliece," Accessed: Jan. 13, 2025. [Online]. Available: <https://classic.mceliece.org/>
- [19] Microchip Technology Inc., "32-bit MCUs for Automotive," Accessed: Jan. 13, 2025. [Online]. Available: <https://www.microchip.com/en-us/solutions/automotive-and-transportation/automotive-products/microcontrollers-and-microprocessors/32-bit-mcus>
- [20] Infineon Technologies AG, "32-bit AURIX™ TriCore™ microcontroller," Accessed: Jan. 13, 2025. [Online]. Available: <https://www.infineon.com/cms/en/product/microcontroller/32-bit-tricore-microcontroller/>